



CENG376 – COMPUTER GRAPHICS

Final Project Report

Project Name: Pitstop Challenge

Group Members:

- Salih Akoğlu (23118080072)

- Ediz Gün (23118080082)

1. Project Overview

The "Pitstop Challenge" is an interactive 3D Formula 1 pit garage simulation developed strictly using WebGL. In this simulation, an F1 car is positioned in the pit lane, and the user navigates freely through the garage environment to perform component replacements. The user acts as a pit crew member, picking up loose components (tires and front wing) and attaching them to the vehicle. The project successfully demonstrates hierarchical modeling, proximity-based snapping, dynamic lighting, and procedural texture mapping entirely through low-level WebGL and GLSL.

2. Fulfillment of Project Requirements

2.1. 3D Scene and Camera Controls

The project features a fully 3D environment, moving away from 2D or strict bird's-eye views. The camera is implemented as a free-roaming first-person camera. Users can move freely along all three axes (X, Y, Z) using the **WASD** keys and rotate their view in three dimensions using **mouse look**. This fulfills the requirement for a fully 3D scene and 3-axis free camera movement.

2.2. Object Morphologies

The scene contains multiple object types with distinct morphologies:

1. **F1 Car Body:** A complex, multi-part mesh representing the chassis.
2. **Tires:** Cylindrical shapes with specific radius and depth parameters.
3. **Garage Environment:** Rectangular and planar morphologies forming the walls, floors, and toolboxes.

2.3. Texture Mapping

To enhance visual realism and meet the project requirements, distinct procedural textures are applied to different objects:

- **Carbon-Fiber/Metallic Paint:** Applied to the F1 car body to simulate the material of a racing car.
- **Matte Rubber Texture:** Applied to the cylindrical tires, giving them a realistic rubber look with markings.
- **Concrete/Asphalt & Wall Textures:** Applied to the garage floor and walls for environmental immersion.

2.4. Independently Controllable Objects

Three independently controllable objects/components exist in the scene, which the user can interact with directly:

1. **Pit Stop Components (Tires / Front Wing):** The user can pick up tires or the front wing from the floor (`E` key) and carry them. Through proximity-based distance checks, bringing them close to their mounting points snaps them into place.
2. **Steering Wheel (Cockpit View):** By switching to the cockpit view (`C` key), the user can rotate the steering wheel. Due to parent-child hierarchical modeling, the front tires are linked as child nodes to the steering wheel, meaning their yaw rotation perfectly synchronizes with the user's steering input.
3. **Drag Reduction System (DRS) Flap:** The rear wing flap can be toggled using the `X` key. Modeled as a separate mesh segment, it rotates along its local X-axis to open or close, demonstrating independent local transformations.

2.5. Lighting and Shading

The lighting is implemented using the Phong reflection model in GLSL fragment shaders. The scene includes a main garage spotlight.

- The light source moves with the camera or can be dynamically adjusted.
- The user can increase or decrease the brightness (power) of the flashlight using the `Q` and `R` keys. This directly affects the specular highlights and shading on the car's metallic surfaces in real-time.

3. Organization and Technical Details

The project was developed without the use of external game engines like Unity or high-level libraries like Three.js. It purely relies on fundamental WebGL API, GLSL shaders, and the matrix vector library `MV.js`. The hierarchical transformations are handled via a custom

matrix stack pipeline, ensuring proper parent-to-child local transformations (e.g., car body -> steering wheel -> tires).

4. Kaynakça (References)

1. **Angel, E., & Shreiner, D. (2014).** *Interactive Computer Graphics: A Top-Down Approach with WebGL*. Pearson.
2. **Mozilla Developer Network (MDN).** *WebGL API Documentation and Tutorials*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API
3. **WebGL Fundamentals.** *Lighting, Textures, and Scene Graphs in WebGL*. Retrieved from <https://webglfundamentals.org/>
4. **Course Materials:** `MV.js` and `webgl-utils.js` utility libraries provided during the Computer Graphics course for matrix operations and WebGL context initialization.